EJAVA Unit-3

1. The Collection Framework [CHAPTER 1 Collection Framework]

The Collection Framework in Java is a **framework that provides an architecture to store and manipulate a group of objects**. It performs operations like **searching, sorting, insertion, deletion, and manipulation of data**. It provides many interfaces and classes in the java.util package.

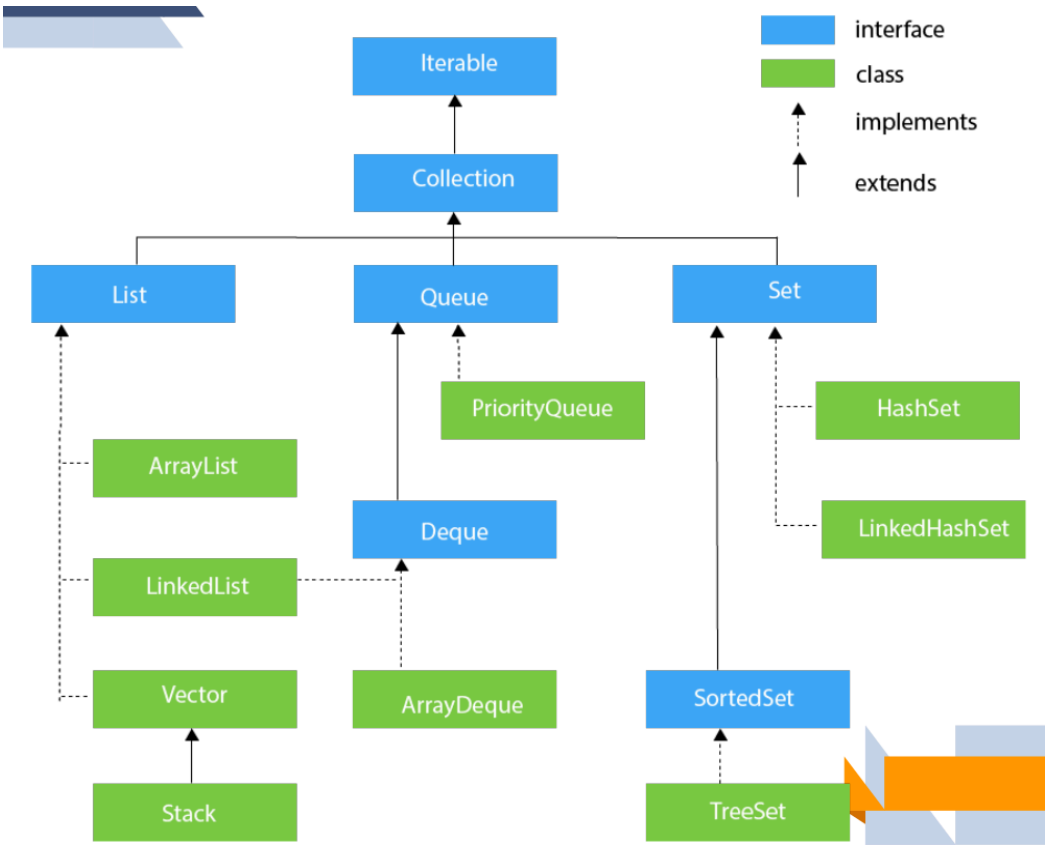
Main Interfaces:

- List
- Set
- Queue
- Deque

Main Classes:

- ArrayList
- LinkedList
- Vector
- PriorityQueue
- HashSet
- LinkedHashSet
- TreeSet

Hierarchy (brief):



Package used:

import java.util.*;

2. The List Interface

The List interface is a **child interface of Collection interface**. It stores an **ordered collection of objects** and allows **duplicate elements**. It is implemented by classes such as **ArrayList, LinkedList, Vector, and Stack**.

Points:

- Maintains insertion order
- Allows duplicate values
- Elements can be accessed using index

Syntax:
List<String> list = new ArrayList<>();

Implementing classes:

- ArrayList
- LinkedList
- Vector
- Stack

3. The List Interface Methods

Method	Description
add()	Adds an element to the list
addAll()	Adds all elements of one list to another
get()	Accesses element using index
iterator()	Returns iterator to access elements
set()	Updates an element
remove()	Removes an element
removeAll()	Removes all elements
clear()	Removes all elements from list

size()	Returns number of elements
toArray()	Converts list into array
contains()	Checks whether an element exists

4. The ArrayList Class

ArrayList is a class that **implements the List interface**. It uses a **dynamic array** to store elements. It **maintains insertion order**, allows **duplicate elements**, and supports **random access** of elements. It is **non-synchronized**.

Features:

- Uses dynamic array
- Allows duplicate values
- Maintains insertion order
- Non-synchronized
- Supports random access

Simple Example:

```
import java.util.*;

class Test{
   public static void main(String args[]){

      ArrayList<String> list=new ArrayList<>();

      list.add("Java");
      list.add("Python");

      System.out.println(list);
   }
}
```

Output:

```
[Java, Python]
```

5. Operations in a Java ArrayList Interface

1. **Adding elements** → add()
2. **Updating elements** → set()
3. **Searching elements** → indexOf(), lastIndexOf()
4. **Removing elements** → remove()
5. **Accessing elements** → get()
6. **Checking element presence** → contains()

Example:

```
ArrayList<String> list = new ArrayList<>();

list.add("Java");     // add
list.set(0,"Python");  // update
list.get(0);          // access
list.indexOf("Python");// search
list.remove("Python"); // remove
list.contains("Java"); // check
```

6. The LinkedList Class

LinkedList is a class that **implements the Collection/List interface** and uses a **doubly linked list** internally to store elements. It **maintains insertion order**, allows **duplicate elements**, and is **non-synchronized**.

Features:

- Uses doubly linked list
- Allows duplicate values
- Maintains insertion order
- Non-synchronized
- Better for insertion and deletion operations

Simple Example:

```
import java.util.*;

class Test{
   public static void main(String args[]){
      LinkedList<String> list=new LinkedList<>();
      list.add("Java");
      list.add("Python");
      System.out.println(list);
```

```
    }
}
```

Output:

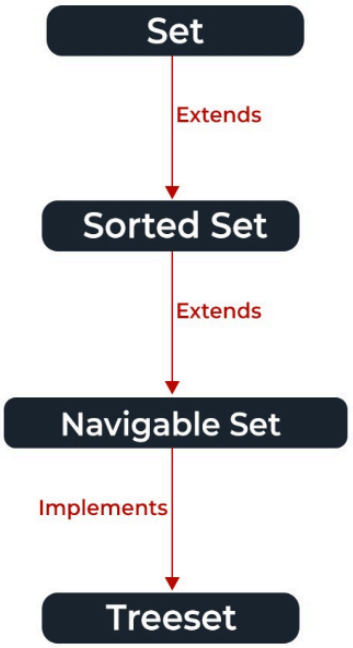
[Java, Python]

7. The LinkedList Class Vs ArrayList class

LinkedList	ArrayList
Uses **doubly linked list** internally	Uses **dynamic array** internally
Better for **insertion and deletion**	Better for **searching and accessing**
Accessing elements is **slower**	Accessing elements is **faster**
Memory usage is **more**	Memory usage is **less**
Elements are stored in **different memory locations**	Elements are stored in **continuous memory locations**
Random access is **not efficient**	Random access is **efficient**
Manipulation is **faster**	Manipulation is **slower**
Implements **List and Deque interfaces**	Implements only **List interface**

8. The Set Interface

The Set interface is present in the java.util package and **extends the Collection interface**. It stores an **unordered collection of objects** and **does not allow duplicate elements**.



Features:

- Does not allow duplicate values
- Stores unique elements
- Unordered collection
- Supports mathematical set operations
- Implemented by HashSet, LinkedHashSet, TreeSet

Syntax:

```
Set<String> set = new HashSet<>();
```

Method	Description
add(element)	Adds an element to the set if it is not already present.
addAll(collection)	Adds all elements from another collection to the set.
clear()	Removes all elements from the set.
contains(element)	Checks whether an element exists in the set.
containsAll(collection)	Checks whether the set contains all elements of a collection.
hashCode()	Returns the hash code value of the set.
isEmpty()	Checks whether the set is empty or not.
iterator()	Returns an iterator to traverse elements in the set.
remove(element)	Removes a specific element from the set.
removeAll(collection)	Removes all elements of the specified collection from the set.
retainAll(collection)	Retains only the specified elements in the set.
size()	Returns the number of elements in the set.
toArray()	Converts the set into an array.

9. The Set Interface – Creating Set Objects

Since Set is an interface, **objects cannot be created directly**. We need classes like HashSet or LinkedHashSet to create Set objects.

Syntax:
Set<String> set1 = new HashSet<>();
Set<String> set2 = new LinkedHashSet<>();

Simple Example:
Set<String> set = new HashSet<>();
set.add("Java");
set.add("Python");
System.out.println(set);

Output:
[Java, Python]

10. Operations on the Set Interface

1. **Intersection** → Returns common elements from two sets
2. **Union** → Combines elements of both sets
3. **Difference** → Removes elements of one set from another set

Example:

```
Set<Integer> s1 = new HashSet<>(Arrays.asList(1,2,3));
Set<Integer> s2 = new HashSet<>(Arrays.asList(2,3,4));

s1.retainAll(s2); // Intersection
s1.addAll(s2);    // Union
s1.removeAll(s2); // Difference
```

11. List Vs Set Interface

List	Set
Ordered collection	Unordered collection
Allows duplicate elements	Does not allow duplicate elements
Elements can be accessed using index	Index access is not available
Multiple null values can be stored	Only one null value can be stored
Implemented by ArrayList, LinkedList, Vector, Stack	Implemented by HashSet, LinkedHashSet, TreeSet

12. Vector Class

Vector is a class that **implements the List interface** and uses a **dynamic array** to store elements. It is **synchronized (thread-safe)** and maintains **insertion order**. It is similar to ArrayList, but slower because of synchronization.

Features:

- Dynamic size
- Thread-safe (synchronized)
- Maintains insertion order
- Allows duplicate and null values
- Legacy class

Syntax:

```
Vector<String> v = new Vector<>();
```

Simple Example:

```
import java.util.*;

class Test{
  public static void main(String args[]){

    Vector<String> v = new Vector<>();

    v.add("Java");
    v.add("Python");

    System.out.println(v);
  }
}
```

Output:

```
[Java, Python]
```

13. Key Features of Vector

- **Dynamic Size** → Automatically grows when elements are added
- **Synchronized** → Thread-safe, only one thread can access at a time
- **Maintains insertion order**
- **Allows duplicate and null values**
- **Legacy class** but compatible with Collection framework
- **Supports Enumeration and Iterator** for traversing elements

14. Common methods in Vector

Common methods in Vector

Method	Description
add()	Adds an element to the vector
addElement()	Adds an element at the end

get()	Returns element at specified index
set()	Replaces an element
remove()	Removes an element
removeElement()	Removes a specific element
size()	Returns number of elements
capacity()	Returns current capacity of vector
contains()	Checks whether element exists
clear()	Removes all elements
firstElement()	Returns first element
lastElement()	Returns last element

15. Stack Class

Stack is a class in the Java Collection Framework that follows the **LIFO (Last In First Out)** principle. It extends the **Vector class** and provides special methods for stack operations.

Features:

- Follows **LIFO**
- Extends Vector
- Allows duplicate elements
- Provides stack-specific operations

Common methods:

- push() → Inserts element into stack
- pop() → Removes top element
- peek() → Returns top element
- empty() → Checks whether stack is empty
- search() → Searches an element

Syntax:

Stack<Integer> s = new Stack<>();

Simple Example:

```
Stack<Integer> s = new Stack<>();

s.push(10);
s.push(20);

System.out.println(s.pop());
```

Output:

20

16. Methods in Stack

Method	Description
push()	Inserts an element into the stack
pop()	Removes and returns the top element
peek()	Returns the top element without removing it
empty()	Checks whether the stack is empty
search()	Searches for an element in the stack
size()	Returns number of elements in the stack
clear()	Removes all elements from the stack
contains()	Checks whether an element exists

17. Multithreading [Chapter 2-Multithreaded Programming]

Multithreading in Java is the process of **executing multiple threads simultaneously**. A thread is a **lightweight sub-process** and the smallest unit of processing. It is used to achieve **multitasking** efficiently because threads share the same memory area.

Advantages of Multithreading:

- Does not block the user because threads are independent
- Multiple operations can be performed at the same time

- Saves execution time
- Exception in one thread does not affect other threads

Simple Example:

```
class MyThread extends Thread{

  public void run(){
    System.out.println("Thread running...");
  }
}

class Test{
  public static void main(String args[]){
    MyThread t = new MyThread();
    t.start();
  }
}
```

Output:

Thread running...

18. Advantages of Java Multithreading

1. **Does not block the user** → Threads are independent
2. **Performs multiple operations simultaneously** → Saves execution time
3. **Exception in one thread does not affect other threads**
4. **Threads share memory** → Saves memory space
5. **Context switching is faster** than processes

19. Multitasking

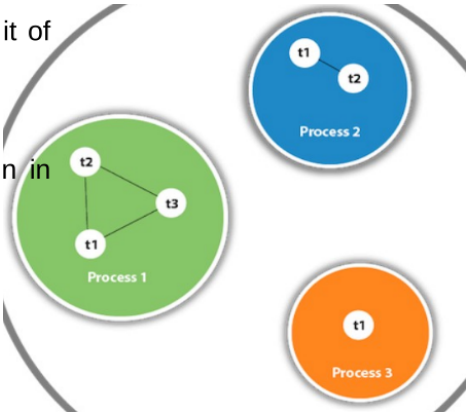
Multitasking is the process of **executing multiple tasks simultaneously** to utilize CPU efficiently. It can be achieved in **two ways**:

1. **Process-based Multitasking (Multiprocessing)**
    - Each process has separate memory
    - Process is heavyweight
    - Communication cost is high
2. **Thread-based Multitasking (Multithreading)**
    - Threads share same memory

- Thread is lightweight
- Communication cost is low

20. Java Thread Model

A thread is a **lightweight subprocess** and the **smallest unit of processing**. It is a separate path of execution inside a process. Multiple threads can exist inside one process and they **share the same memory area**.

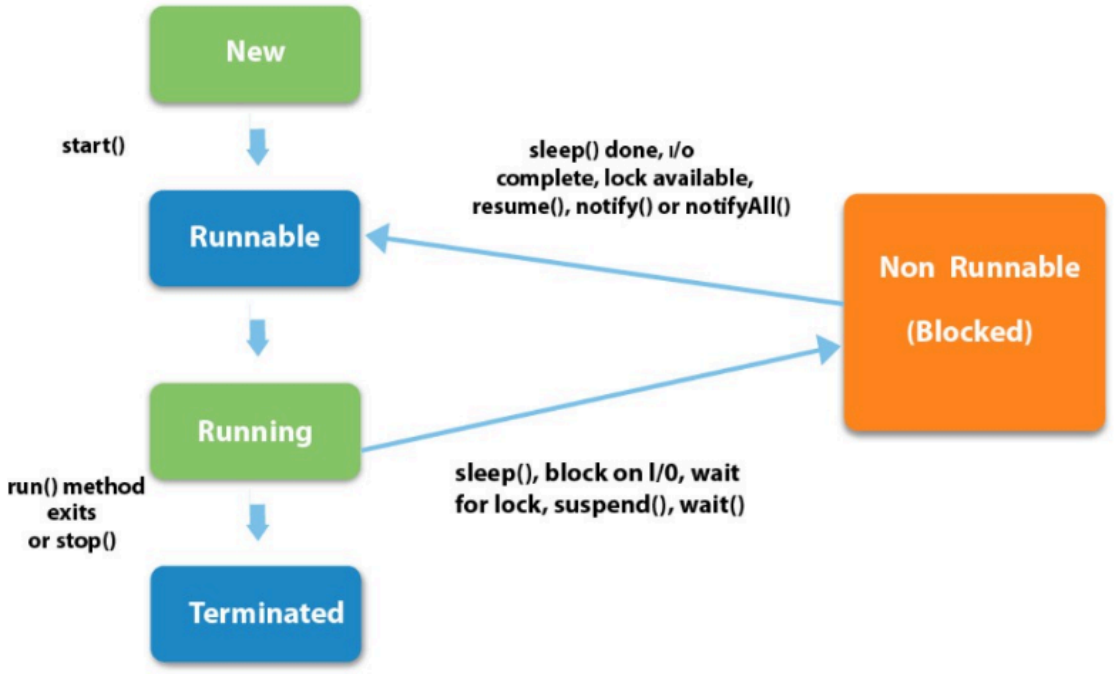


Points:

- Threads are independent
- Uses shared memory area
- Executed inside a process
- Context switching occurs between threads
- One process can have multiple threads

21. Life cycle of a Thread (Thread States)

The life cycle of a thread in Java is controlled by **JVM**. A thread passes through different states during execution.



Thread States:

1. **New** → Thread object is created but start() is not called
2. **Runnable** → start() is called and thread is ready to run

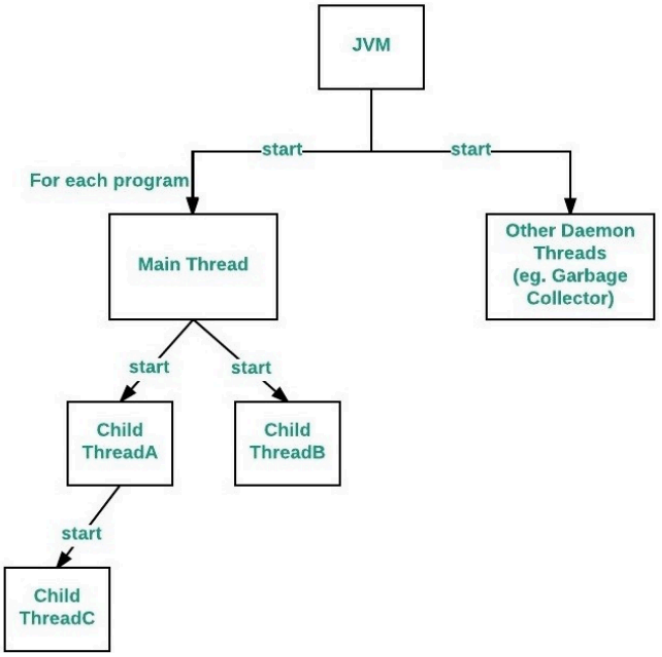
3. **Running** → Thread scheduler selects the thread for execution
4. **Non-Runnable (Blocked)** → Thread is alive but not eligible to run
5. **Terminated** → run() method execution is completed

Flow:

New → Runnable → Running → Blocked → Terminated

22. Main thread in Java

When a Java program starts, one thread begins execution immediately. This is called the **main thread**, because it is the first thread executed by the program. Other child threads are created from the main thread.



Properties:

- Parent thread of all child threads
- Executes when the program starts
- Usually finishes execution last
- Performs shutdown actions

To get reference of main thread:

Thread t = Thread.currentThread();

23. Main thread Example

```
class Test{
  public static void main(String args[]){

    Thread t = Thread.currentThread();

    System.out.println(t);
  }
}
```

Output:

Thread[main,5,main]

Here currentThread() returns the reference of the currently executing thread (main thread).

24. Java Thread Class

Java provides the Thread class to achieve **thread programming**. It provides constructors and methods to create and perform operations on threads. The Thread class **extends Object class** and **implements Runnable interface**.

Common constructors:

Thread()

Thread(String name)

Thread(Runnable r)

Thread(Runnable r, String name)

Simple Example:

```
class MyThread extends Thread{
  public void run(){
    System.out.println("Thread running");
  }
}

class Test{
  public static void main(String args[]){
    MyThread t = new MyThread();
    t.start();
  }
}
```

Output:

Thread running

25. Java Thread class Methods

Modifier and Type	Method	Description
void	start()	Starts the execution of the thread

void	run()	Performs action for a thread
static void	sleep()	Pauses thread for specified time
static Thread	currentThread()	Returns reference of currently executing thread
void	join()	Waits for a thread to die
int	getPriority()	Returns thread priority
void	setPriority()	Changes thread priority
String	getName()	Returns thread name
void	setName()	Changes thread name
long	getId()	Returns thread ID
boolean	isAlive()	Checks whether thread is alive
static void	yield()	Pauses current thread and allows other threads to execute
void	suspend()	Suspends the thread
void	resume()	Resumes suspended thread
void	stop()	Stops the thread
void	destroy()	Destroys thread group and subgroups
void	notify()	Gives notification to one waiting thread
void	notifyAll()	Gives notification to all waiting threads

26.How to create thread

There are **two ways** to create a thread in Java:

1. **By implementing Runnable interface**
2. **By extending Thread class**

Syntax:

```
// Runnable
class MyThread implements Runnable
```

```
// Thread class
class MyThread extends Thread
```

Main Steps:

- Create thread class
- Override run() method
- Create object
- Call start() method

1) By implementing Runnable interface

```
class MyThread implements Runnable{

   public void run(){
      System.out.println("Thread is running...");
   }
}

class Test{
   public static void main(String args[]){

      MyThread m = new MyThread();

      Thread t = new Thread(m);

      t.start();
   }
}
```

Output:

```
Thread is running...
```

2) By extending Thread class

```
class MyThread extends Thread{

   public void run(){
      System.out.println("Thread is running...");
   }
}
```

```
class Test{
  public static void main(String args[]){

    MyThread t = new MyThread();

    t.start();
  }
}
```

Output:

Thread is running...

27. Implementing the Runnable Interface

The Runnable interface is used to create a thread. It contains only one method called run(). After implementing Runnable, we need to create a Thread object and call start() method.

Steps:

1. Implement Runnable interface
2. Override run() method
3. Create object of class
4. Pass object to Thread class
5. Call start()

Example:

```
class MyThread implements Runnable{

  public void run(){
    System.out.println("Concurrent thread started...");
  }
}

class Test{
  public static void main(String args[]){

    MyThread m = new MyThread();

    Thread t = new Thread(m);
```

```
    t.start();
  }
}
```

Output:

Concurrent thread started...

27. Extending Thread Class

Another way to create a thread is by **extending the Thread class**. The class must override the run() method, which contains the code to be executed by the thread. Then call start() to begin execution.

Steps:

1. Extend Thread class
2. Override run() method
3. Create object of class
4. Call start() method

Example:

```
class MyThread extends Thread{

  public void run(){
    System.out.println("Concurrent thread started...");
  }
}

class Test{
  public static void main(String args[]){

    MyThread t = new MyThread();

    t.start();
  }
}
```

Output:

Concurrent thread started...

28. isAlive() and join() methods of Thread Class

isAlive()

isAlive() checks whether a thread is **alive (running)** or not. It returns **true** if the thread has started and not yet terminated; otherwise returns **false**.

Syntax:

```
boolean b = t.isAlive();
```

join()

join() is used to make one thread **wait until another thread finishes execution**.

Syntax:

```
t.join();
```

Example:

```
class MyThread extends Thread{

  public void run(){
    System.out.println("Thread running");
  }
}

class Test{
  public static void main(String args[]) throws Exception{

    MyThread t = new MyThread();

    System.out.println(t.isAlive());

    t.start();

    t.join();

    System.out.println(t.isAlive());
  }
```

```
}
```

Output:

```
false
Thread running
false
```

29. Synchronization

Synchronization in Java is the process of **controlling access of multiple threads to a shared resource**. It allows **only one thread at a time** to access the shared resource and prevents inconsistency problems.

Why synchronization is used:

- Prevents **thread interference**
- Prevents **data inconsistency**
- Protects shared resources

Syntax:

```
synchronized void display(){
  // code
}
```

30. Types of Thread Synchronization

There are **two types of thread synchronization**:

1. **Mutual Exclusive**
    - Prevents threads from interfering with each other while sharing data
    - Types:
        - Synchronized method
        - Synchronized block
2. **Cooperation (Inter-thread Communication)**
    - Allows synchronized threads to communicate with each other
    - Uses:
        - wait()
        - notify()
        - notifyAll()

31. Understanding the problem without Synchronization

If multiple threads access the **same shared resource simultaneously** without synchronization, it may produce **incorrect or inconsistent results** because threads interfere with each other.

Example:

```
class Table{

  void print(int n){

    for(int i=1;i<=3;i++)
      System.out.println(n*i);
  }
}

class T1 extends Thread{
  Table t = new Table();

  public void run(){
    t.print(5);
  }
}

class T2 extends Thread{
  Table t = new Table();

  public void run(){
    t.print(10);
  }
}

class Test{
  public static void main(String args[]){

    new T1().start();
    new T2().start();
  }
}
```

Possible Output:

```
5
10
10
20
15
30
```

The output may become mixed because both threads execute simultaneously without synchronization.

32. Java synchronized method

If a method is declared using the synchronized keyword, it is called a **synchronized method**. It is used to **lock an object for a shared resource**, allowing only one thread to access it at a time.

Syntax:

```
synchronized void method(){
  // code
}
```

Example:

```
class Table{

   synchronized void print(int n){

      for(int i=1;i<=3;i++)
         System.out.println(n*i);
   }
}

class MyThread extends Thread{

   Table t;

   MyThread(Table t){
      this.t=t;
   }

   public void run(){
      t.print(5);
```

```
  }
}

class Test{
  public static void main(String args[]){

    Table obj=new Table();

    new MyThread(obj).start();
    new MyThread(obj).start();
  }
}
```

Point:

- Only one thread can execute the synchronized method at a time.
- Lock is automatically released after method execution completes.

33. Using Synchronized block

A synchronized block is used when **only a specific part of a method needs synchronization**. It locks only the block instead of the entire method, improving performance.

Syntax:

```
synchronized(object){
  // code
}
```

Example:

```
class Table{

  void print(int n){

    synchronized(this){

      for(int i=1;i<=3;i++)
        System.out.println(n*i);
    }
  }
}
```

```
class Test{
  public static void main(String args[]){

    Table t = new Table();

    new Thread(()->t.print(5)).start();
    new Thread(()->t.print(10)).start();
  }
}
```

Point:

- Synchronizes only required code
- Better performance than synchronized method

34. Difference between synchronized keyword and synchronized block

Synchronized Method	Synchronized Block
Locks the **entire method**	Locks **only a specific block of code**
Uses method declaration	Uses synchronized(object) block
Lower performance because whole method is locked	Better performance because only required code is locked
No other thread can access any synchronized method of the object	Other threads can access remaining code outside the block
Used when complete method needs synchronization	Used when only part of method needs synchronization

35. Inter-thread Communication

Inter-thread communication (Co-operation) is a mechanism that allows **synchronized threads to communicate with each other**. It allows one thread to pause execution and another thread to enter the same critical section.

Methods used:

- wait() → Makes thread wait until another thread notifies it

- notify() → Wakes up one waiting thread
- notifyAll() → Wakes up all waiting threads

Note: These methods can be called only inside a **synchronized block/method**.

36. Understanding the process of Inter-thread Communication

Inter-thread communication allows threads to **coordinate and communicate with each other** while sharing resources. One thread waits and another thread sends notification for execution. It is achieved using wait(), notify(), and notifyAll() methods.

Process:

1. Thread enters synchronized block
2. Thread calls wait() and releases lock
3. Another thread enters same block
4. Another thread calls notify() or notifyAll()
5. Waiting thread wakes up and continues execution

Flow:

Thread → wait()
↓
Waiting state
↓
notify()/notifyAll()
↓
Thread resumes execution

37. Differences between wait() and sleep()

wait()	sleep()
wait() method **releases the lock**	sleep() method **does not release the lock**
Method of **Object class**	Method of **Thread class**
Non-static method	**Static** method
Used for **inter-thread communication**	Used to **pause thread execution**
Thread should be notified using **notify() / notifyAll()**	Thread resumes automatically after specified time

Must be called inside **synchronized block/method**	No such requirement
Generally used based on a **condition**	Simply used to put thread into **sleep state**

38. Java Applet [Chapter 3 - Event Handling]

Applet is a **special type of Java program embedded in a webpage** to generate dynamic content. It runs inside the **browser** and works at the **client side**.

Advantages:

- Works at client side → less response time
- Secure
- Platform independent

Drawback:

- Requires browser plugin to run

Simple Example:

```
import java.applet.*;
import java.awt.*;

public class MyApplet extends Applet{

  public void paint(Graphics g){
    g.drawString("Hello Applet",50,50);
  }
}
```

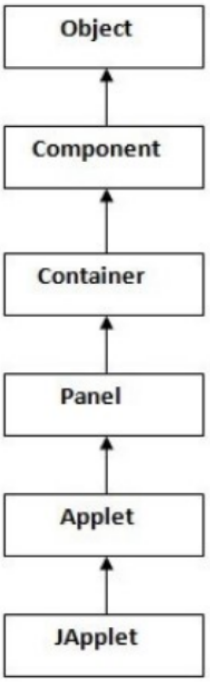
39. Differences between an Applet and a stand-alone Java application

Applet	Stand-alone Java Application
Does not use main() method	Uses main() method
Embedded inside an **HTML page**	Runs independently

Runs inside a **browser or applet viewer**	Runs directly using JVM
Downloaded from web page	Executed from local system
Requires browser support/plugin	Does not require browser
Life cycle controlled by browser	Execution controlled by program

40. Hierarchy of Applet

- Object → Root class of Java
- Component → Base class for GUI components
- Container → Holds components
- Panel → Groups components
- Applet → Used for applet programming
- JApplet → Swing version of Applet



41. Applet Basics

Applets are **small applications** that are accessed through an Internet server, transported over the Internet, automatically installed, and run as part of a web document. Applets have **limited access to system resources** for security purposes.

Important imports:

```
import java.awt.*;
import java.applet.*;
```

Purpose:

- java.awt.* → Imports AWT classes for GUI interaction
- java.applet.* → Imports Applet class

Point:
All applets created must be **subclasses of Applet class**.

42. Applet Life Cycle

The Applet life cycle defines the **sequence of methods called during applet execution**. JVM invokes these methods automatically.

Life cycle stages:

1. **init()** → Initializes the applet *(called once)*
2. **start()** → Starts applet execution
3. **paint()** → Displays output on screen
4. **stop()** → Stops applet execution
5. **destroy()** → Destroys the applet *(called once)*

Flow:

init()
↓
start()
↓
paint()
↓
stop()
↓
destroy()

43. Life cycle methods for Applet

The Applet class provides **4 life cycle methods**, and Component class provides **1 method (paint())**. These methods are called automatically by JVM during applet execution.

Method	Description
init()	Initializes the applet, called only once
start()	Starts applet execution
paint()	Displays output on screen

stop()	Stops applet execution
destroy()	Destroys applet and releases resources

44. AWT Event Handling

AWT Event Handling is the mechanism that **controls events and decides what action should be performed when an event occurs**. Java uses the **Delegation Event Model** to handle events.

Points:

- Event → Change in state of an object
- Event Handler → Code executed when an event occurs
- Uses Delegation Event Model
- Handles user actions like button click, keyboard input, mouse movement, etc.

45. Types of Events

Events are broadly classified into **two types**:

1. **Foreground Events**
    - Require direct interaction of user
    - Generated by GUI actions
    - Examples:
        - Button click
        - Mouse movement
        - Keyboard input
        - Selecting items
2. **Background Events**
    - Do not require direct user interaction
    - Generated automatically by system
    - Examples:
        - Timer expiration
        - Hardware/software failure
        - Operation completion
        - Operating system interrupts

46. Event Handling

Event Handling is a mechanism that **controls events and decides what action should happen when an event occurs**. The code that executes when an event occurs is called an **event handler**. Java uses the **Delegation Event Model** for handling events.

Points:

- Controls and handles events
- Event handler executes when an event occurs
- Uses Delegation Event Model
- Responds to user actions like button click, mouse movement, keyboard input, etc.

47. Delegation Event Model

The Delegation Event Model is a mechanism used by Java to **generate and handle events**. In this model, the **source generates the event** and the **listener handles the event**. The listener must be registered with the source object.

Components:

1. **Source**
    - Object on which event occurs
    - Generates event information
2. **Listener**
    - Also called event handler
    - Receives and processes events

Flow:

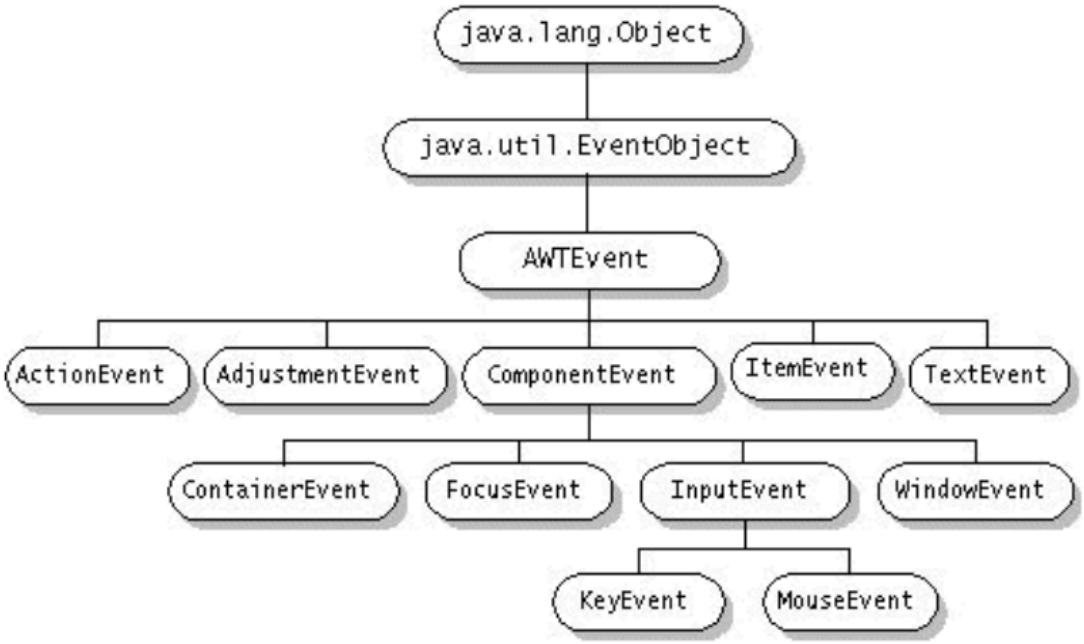
Source → Event → Listener

48. Event Classes

Event classes represent **different types of events generated by user interactions** in GUI components. The java.awt.event package provides various event classes.

Event Class	Description
ActionEvent	Generated when button is clicked
MouseEvent	Generated when mouse action occurs

KeyEvent	Generated when keyboard key is pressed/released
ItemEvent	Generated when item selection changes
FocusEvent	Generated when component gains or loses focus
WindowEvent	Generated when window state changes
TextEvent	Generated when text value changes



49. Event Classes

The AWTEvent class is defined in the java.awt package and acts as the **root class of all event classes**. The java.awt.event package provides different event classes generated by GUI components.

Event Class	Description
ActionEvent	Generated when a button is pressed, list item is double-clicked, or menu item is selected
AdjustmentEvent	Generated when a scroll bar is manipulated
ComponentEvent	Generated when a component is hidden, moved, resized, or becomes visible
ContainerEvent	Generated when a component is added to or removed from a container
FocusEvent	Generated when a component gains or loses keyboard focus

InputEvent	Abstract superclass for all component input event classes
ItemEvent	Generated when a checkbox or list item is clicked or selected/deselected
KeyEvent	Generated when input is received from the keyboard
MouseEvent	Generated when mouse is moved, dragged, clicked, pressed, released, enters, or exits a component
MouseWheelEvent	Generated when a mouse wheel is moved
TextEvent	Generated when value of text area or text field changes
WindowEvent	Generated when a window is activated, closed, opened, deactivated, iconified, deiconified, or quit

49. Sources of Events

Event Source	Description
Button	Generates action events when the button is pressed
CheckBox	Generates item events when checkbox is selected or deselected
Choice	Generates item events when the choice is changed
List	Generates action events when an item is double-clicked and item events when selected/deselected
MenuItem	Generates action events when menu item is selected and item events for checkable menu items
ScrollBar	Generates adjustment events when scroll bar is manipulated
Text Components	Generates text events when user enters a character
Window	Generates window events when window is activated, closed, deactivated, deiconified, iconified, opened, or quit

50. Event Listener Interfaces

Interface	Description
ActionListener	Defines one method to receive action events
AdjustmentListener	Defines one method to receive adjustment events
ComponentListener	Defines four methods to recognize when a component is hidden, moved, resized, or shown
ContainerListener	Defines two methods to recognize when a component is added or removed from a container
FocusListener	Defines two methods to recognize when a component gains or loses keyboard focus
ItemListener	Defines one method to recognize when the state of an item changes
KeyListener	Defines three methods to recognize when a key is pressed, released, or typed
MouseListener	Defines five methods to recognize when mouse is clicked, enters, exits, pressed, or released
MouseMotionListener	Defines two methods to recognize when mouse is dragged or moved
MouseWheelListener	Defines one method to recognize when mouse wheel is moved
TextListener	Defines one method to recognize when text value changes
WindowFocusListener	Defines two methods to recognize when a window gains or loses input focus
WindowListener	Defines seven methods to recognize when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit

51. Java Event classes and Listener interfaces

Event Class	Listener Interface
ActionEvent	ActionListener

MouseEvent	MouseListener, MouseMotionListener
MouseWheelEvent	MouseWheelListener
KeyEvent	KeyListener
ItemEvent	ItemListener
TextEvent	TextListener
AdjustmentEvent	AdjustmentListener
WindowEvent	WindowListener
ComponentEvent	ComponentListener
ContainerEvent	ContainerListener
FocusEvent	FocusListener

52. Using the Delegation Event Model

Using the Delegation Event Model follows **two steps**:

1. **Implement the appropriate listener interface**
    - The listener receives the required event type.
2. **Register the listener with the source object**
    - Allows the listener to receive event notifications.

Points:

- One source can generate multiple events
- Each event must be registered separately
- One listener can handle multiple event types by implementing required interfaces

53. Swings

Swing is used to **create window-based applications** in Java. It provides **platform-independent and lightweight GUI components** through the javax.swing package.

Features:

- Platform independent
- Lightweight components

- Used for GUI applications
- Provides rich controls

Common Swing Components:

- JLabel
- JButton
- JTextField
- JTextArea
- JRadioButton
- JCheckBox
- JMenu

Package:

import javax.swing.*;

54. JApplet Class

JApplet is a **Swing class that extends the Applet class**. It is used to create **Swing-based applets** and provides support for components like **content pane, glass pane, and root pane**.

Features:

- Extends Applet class
- Part of javax.swing package
- Supports Swing components
- Uses content pane to add components

Syntax:

```
import javax.swing.*;

public class MyApplet extends JApplet{
}
```

Adding component:

add(component);

55. Java JFrame

JFrame is a class in javax.swing package used to **create and manage top-level windows** in Java GUI applications. It acts as the **main window/container** for Swing applications.

Features:

- Creates window-based applications
- Platform independent
- Supports GUI components
- Provides title bar, buttons, and borders

Simple Example:

```
import javax.swing.*;

class Test{
  public static void main(String args[]){

    JFrame f = new JFrame("My Frame");

    f.setSize(300,200);
    f.setVisible(true);
  }
}
```

56. Methods of Java JFrame

Method	Description
setTitle(String title)	Sets the title of the JFrame
setSize(int width, int height)	Sets the size of the JFrame
setDefaultCloseOperation(int operation)	Sets the default close operation of JFrame (EXIT_ON_CLOSE, HIDE_ON_CLOSE, DO_NOTHING_ON_CLOSE)
setVisible(boolean b)	Sets visibility of JFrame (true → visible, false → hidden)
setLayout(LayoutManager manager)	Sets layout manager for arranging components
add(Component comp)	Adds a component to JFrame

remove(Component comp)	Removes a component from JFrame
validate()	Recalculates the layout of components
setResizable(boolean resizable)	Controls whether user can resize JFrame
setIconImage(Image image)	Sets icon image for JFrame window

57. JApplets Vs JFrame

Feature	JApplet	JFrame
Purpose	Used for creating Java applets	Used for creating standalone GUI applications
Deployment	Embedded within web browsers	Runs independently of web browsers
Lifecycle	Managed by browser or applet viewer	Controlled by programmer code
HTML Embedding	Embedded within HTML using <applet> tag	Not embedded within HTML
Window Decorations	Typically no window decorations	Includes title bar, borders, and control buttons
Usage	Less used due to security concerns and alternatives	Commonly used for modern GUI applications

58. Java JLabel

JLabel is a Swing component used to **display a single line of read-only text, image, or both** in a container. Users **cannot directly edit** the text. It inherits the JComponent class.

Features:

- Displays text or images
- Read-only component
- Supports text alignment
- Inherits JComponent

Common Constructors:

JLabel()

JLabel(String s)

JLabel(Icon i)

JLabel(String s, Icon i, int alignment)

Simple Example:

```
import javax.swing.*;

class Test{
  public static void main(String args[]){

    JFrame f = new JFrame();

    JLabel l = new JLabel("Hello Java");

    f.add(l);
    f.setSize(300,200);
    f.setVisible(true);
  }
}
```

59. Commonly used Constructors of Java JLabel

Constructor	Description
JLabel()	Creates a JLabel with no image and empty text
JLabel(String s)	Creates a JLabel with specified text
JLabel(Icon i)	Creates a JLabel with specified image
JLabel(String s, Icon i, int horizontalAlignment)	Creates a JLabel with specified text, image, and horizontal alignment

60. Commonly used Methods of Java JLabel

Method	Description
String getText()	Returns the text displayed by the label
void setText(String text)	Sets the text of the label
void setHorizontalAlignment(int alignment)	Sets the alignment of label contents along X-axis
Icon getIcon()	Returns the image/icon displayed by the label
int getHorizontalAlignment()	Returns the alignment of label contents

61. Java JButton

JButton is a Swing component used to **create a labeled button** with platform-independent implementation. When the button is clicked, it performs some **action/event**. It inherits the AbstractButton class.

Features:

- Used to create buttons
- Performs action on click
- Platform independent
- Inherits AbstractButton

Common Constructors:

- JButton()
- JButton(String s)
- JButton(Icon i)

Simple Example:

```
import javax.swing.*;

class Test{
  public static void main(String args[]){
    JFrame f = new JFrame();

    JButton b = new JButton("Click");
    f.add(b);
    f.setSize(300,200);
    f.setVisible(true);
```

```
    }
}
```

62. Common Constructors of Java JButton

Constructor	Description
JButton()	Creates a button with no text and icon
JButton(String s)	Creates a button with specified text
JButton(Icon i)	Creates a button with specified icon object

63. Commonly used Methods of AbstractButton Class

Method	Description
void setText(String s)	Sets specified text on the button
String getText()	Returns the text of the button
void setEnabled(boolean b)	Enables or disables the button
void setIcon(Icon b)	Sets the specified icon on the button
Icon getIcon()	Returns the icon of the button
void setMnemonic(int a)	Sets the mnemonic on the button
void addActionListener(ActionListener a)	Adds an action listener to the button

64. Java JTextField

JTextField is a Swing component used to **edit a single line of text**. It inherits JTextComponent class.

Common Constructors

Constructor	Description
JTextField()	Creates a new JTextField
JTextField(String text)	Creates a JTextField with specified text
JTextField(String text, int columns)	Creates a JTextField with specified text and columns

JTextField(int columns)	Creates an empty JTextField with specified columns

Common Methods

Method	Description
void addActionListener(ActionListener l)	Adds action listener to receive action events
Action getAction()	Returns the currently set action
void setFont(Font f)	Sets the current font
void removeActionListener(ActionListener l)	Removes the specified action listener

65. Java JCheckBox

JCheckBox is a Swing component used to **create a checkbox**. It is used to **turn an option ON (true) or OFF (false)**. It inherits the JToggleButton class.

Common Constructors

Constructor	Description
JCheckBox()	Creates an unselected checkbox with no text or icon
JCheckBox(String s)	Creates an unselected checkbox with text
JCheckBox(String text, boolean selected)	Creates a checkbox with text and selected status
JCheckBox(Action a)	Creates a checkbox using Action properties

Common Methods

Method	Description
AccessibleContext getAccessibleContext()	Returns the AccessibleContext of JCheckBox
protected String paramString()	Returns string representation of JCheckBox

66. Java JRadioButton

JRadioButton is a Swing component used to **create radio buttons**. It allows the user to **select one option from multiple choices**. It inherits the JToggleButton class.

Common Constructors

Constructor	Description
JRadioButton()	Creates an unselected radio button with no text
JRadioButton(String s)	Creates an unselected radio button with specified text
JRadioButton(String s, boolean selected)	Creates a radio button with specified text and selected status

Common Methods

Method	Description
void setText(String s)	Sets specified text on button
String getText()	Returns text of button
void setEnabled(boolean b)	Enables or disables the button
void setIcon(Icon b)	Sets specified icon on button
Icon getIcon()	Returns icon of button
void setMnemonic(int a)	Sets mnemonic on button
void addActionListener(ActionListener a)	Adds action listener to this object

67. The KeyEvent Class

KeyEvent is an event class in the java.awt.event package used to **handle keyboard events**. It is generated when a **key is pressed, released, or typed**.

Types of Key Events

- keyPressed() → Invoked when a key is pressed
- keyReleased() → Invoked when a key is released
- keyTyped() → Invoked when a key is typed

Common Methods

Method	Description
char getKeyChar()	Returns character associated with key
int getKeyCode()	Returns integer key code
String getKeyText(int keyCode)	Returns text description of key
boolean isActionKey()	Checks whether key is an action key

Simple Example

```
import java.awt.event.*;

class Test implements KeyListener{
   public void keyPressed(KeyEvent e){
      System.out.println("Key Pressed");
   }
   public void keyReleased(KeyEvent e){}
   public void keyTyped(KeyEvent e){}
}
```

68. The KeyListener Interface

KeyListener is an interface in the java.awt.event package used to **handle keyboard events**. It receives events when a key is **pressed, released, or typed**.

Methods of KeyListener

Method	Description
void keyPressed(KeyEvent e)	Invoked when a key is pressed
void keyReleased(KeyEvent e)	Invoked when a key is released
void keyTyped(KeyEvent e)	Invoked when a key is typed

Simple Example

```
import java.awt.event.*;
class Test implements KeyListener{
   public void keyPressed(KeyEvent e){
      System.out.println("Pressed");
   }
```

```
  public void keyReleased(KeyEvent e){}

  public void keyTyped(KeyEvent e){}
}
```

69. Adapter Classes

Adapter classes are helper classes provided in Java to **implement listener interfaces partially**. They provide **empty implementations** of listener methods, so the programmer only overrides the required methods. This reduces code size.

Common Adapter Classes

Adapter Class	Used For
KeyAdapter	Keyboard events
MouseAdapter	Mouse events
WindowAdapter	Window events
FocusAdapter	Focus events
ComponentAdapter	Component events
ContainerAdapter	Container events
MouseMotionAdapter	Mouse motion events
HierarchyBoundsAdapter	HierarchyBoundsListener

Example:

```
import java.awt.event.*;
class Test extends KeyAdapter{
  public void keyPressed(KeyEvent e){
    System.out.println("Key Pressed");
  }
}
```

Advantage:
No need to implement all methods of a listener interface. Only required methods are overridden.